契约驱动快速演化（CDRE）

AI 时代的软件工程开发方法论

> Contract-Driven Rapid Evolution
> 人类经营契约，AI 快速演化实现，自动化验证守门。

为什么需要一套新方法论

AI 编程工具（Claude Code、Cursor、Copilot 等）正在深刻改变软件开发的成本结构。写代码不再是瓶颈，但一个根本性的矛盾随之暴露：**AI 拥有极强的局部实现能力，却缺乏大规模项目的整体组织能力。**

这种失衡源于三个结构性约束：

上下文窗口的物理限制。 AI 无法一次性"看见"整个大型项目。即使 200K token 的上下文窗口，对于一个真正的生产系统也远远不够。AI 必须在局部视野下工作，而谁来定义这些局部的边界、接口和契约，就成了最关键的工程问题。

跨时间一致性的缺失。 AI 没有真正的持久记忆，每次对话都是从零开始。一个持续数月的项目所需要的设计决策连贯性，AI 自身无法维持。

实现成本趋近于零，但验证成本不变。 让 AI 重写一个模块可能只需要几分钟，但验证"这个实现是否正确"的成本并没有降低。这完全颠覆了传统软件工程中以"减少返工"为目标的流程设计逻辑。

CDRE 方法论正是为了应对这些新约束而设计的。它不是对传统方法论的否定，而是在新的成本结构下对经典软件工程原则的重新组合。

核心理念

一句话总结

代码是廉价的一次性产物，契约才是核心资产。

三个范式转换

传统范式	CDRE 范式
实现是"构建"——精心编写，小心维护	实现是"搜索"——在可能空间中快速采样，通过验证收敛
文档是沟通工具——给人读的	文档是编程接口——给 AI 读的，精度直接决定产出质量
测试是事后验证——确认代码是对的	测试是事前规约——定义"对"的标准，引导 AI 生成

角色重新定义

在 CDRE 中，人类的角色从"编码者"转变为**架构师 + 规约设计者 + 验证者**的融合体：

- **架构师**：定义系统边界、技术选型、模块分解
- **规约设计者**：为每个模块编写 AI 可理解的工作规约
- **验证者**：设计自动化测试和质量门禁，审查架构一致性

AI 的角色是**核心实现者**——在人类划定的边界内，快速生成满足规约的代码。

四层架构

CDRE 将软件项目组织为四个层次，每一层有不同的变化速率和责任归属。

```
┌──────────────────────────────────────────────────┐
│  契约层 (Contract Layer)                          │
│  变化速率：极慢     责任：人类                     │
│  内容：ADR、接口类型、数据库 Schema、边界图        │
├──────────────────────────────────────────────────┤
│  规约层 (Specification Layer)                     │
│  变化速率：慢       责任：人类                     │
│  内容：模块 Spec、测试规约、Prompt 模板            │
```

```
├──────────────────────────────────────────┤
│  实现层（Implementation Layer）              │
│  变化速率：极快     责任：AI                  │
│  内容：源代码、配置文件                       │
├──────────────────────────────────────────┤
│  验证层（Verification Layer）                │
│  变化速率：慢       责任：自动化 + 人类       │
│  内容：CI Pipeline、类型检查、测试、Review 清单 │
└──────────────────────────────────────────┘
```

第一层：契约层（Contract Layer）

定义：系统的"宪法"，记录了不可轻易更改的架构决策和模块间的通信协议。

包含物：

架构决策记录（ADR）。 每个重要的技术决策都以 ADR 形式记录，包含三个要素：Context（为什么要做这个决策）、Decision（选了什么）、Consequences（代价和收益）。ADR 的价值在于它不仅记录了"选了什么"，更记录了"为什么选"——这对 AI 在正确的约束下工作至关重要。

接口契约。 模块间通信的形式化定义。推荐使用 TypeScript 类型定义、OpenAPI Spec 或 Protobuf Schema。接口契约必须是**机器可读的**——它们不只是文档，还是类型检查器的输入，可以自动验证实现是否符合契约。

数据模型。 数据库 Schema、核心数据结构的定义。与接口契约一样，必须是形式化的、可自动验证的。

系统边界图。 一张简单的图，展示系统由哪些模块组成、模块之间如何通信。不需要精美，ASCII 图就够。其目的是让任何人（包括 AI）在 30 秒内理解系统的整体形状。

设计原则：

- 契约层的文档面向机器可读，优先使用形式化描述（类型定义 > 自然语言）
- 数量少而精，一个中型项目通常 3-5 个 ADR + 1 份接口定义 + 1 份 Schema
- 变更契约层需要慎重考虑，因为它会波及所有下游层

第二层：规约层（Specification Layer）

定义：为每个模块划定的"工作空间"，是人类与 AI 之间的上下文协议。

包含物：

模块规约（Module Spec）。 每个独立模块一份，不超过一页。包含五个要素：职责（这个模块做什么）、依赖（它需要什么）、行为规则（具体的业务逻辑）、约束（性能、安全等非功能需求）、显式排除（这个模块**不做**什么）。显式排除尤其重要——AI 有过度工程化的倾向，明确告诉它"不要做什么"可以显著提高产出质量。

测试规约。 可执行的验收标准。在 CDRE 中，测试不是事后补充的，而是在实现之前就写好的——它们定义了"正确"的含义。测试规约分为三层：E2E 测试定义业务流程的正确性，集成测试定义模块边界的正确性，单元测试可以交给 AI 在实现时自动生成。

Prompt 模板。 可复用的、版本化的 AI 工作指令模板。一个典型的 prompt 模板定义了：需要注入的上下文组合、AI 应遵循的规则、输出的格式要求。Prompt 模板纳入版本管理，随着对 AI 行为的理解而迭代。

设计原则：

- 遵循"AI 犯错就加约束"的原则——一开始写得粗一些，发现 AI 反复犯某类错误，就在对应模块的规约中加入约束
- 测试先于实现编写
- Prompt 模板是一等公民，享受和代码同等的版本管理待遇

第三层：实现层（Implementation Layer）

定义：由 AI 生成的具体代码，是"廉价的一次性产物"。

核心原则：

```
实现层的三条铁律

1. 不满意？重新生成，不要手动修补
   → 修补会让代码偏离规约，积累隐性技术债

2. 每次生成都注入完整上下文
   → AI 没有记忆，上下文不全 = 产出不一致

3. AI 产出的代码不是"你的代码"
   → 不要对它产生感情，它是可以随时替换的
```

工作方式：将契约层（精简版）+ 相关模块规约 + 具体任务描述 + 测试用例，组装成一个完整的 prompt，交给 AI 执行。如果产出不满意，调整规约或 prompt 后重新生成——而不是在 AI 的代码上做局部修改。

关键理念——实现是"搜索"而非"构建"：你不再是精心构建一个唯一的实现，而是在可能的实现空间中，通过 AI 快速采样多个候选方案，然后用测试和人工判断来收敛到最优解。

第四层：验证层（Verification Layer）

定义：质量的最终防线，由自动化检查和人类审查共同组成。

自动化验证 Pipeline（按执行顺序）：

1. **类型检查**——实现是否符合契约层的类型定义？
2. **Lint**——代码风格是否一致？
3. **单元测试**——函数级行为是否正确？
4. **集成测试**——模块间交互是否符合规约？
5. **E2E 测试**——业务流程是否端到端正确？
6. **契约一致性检查**——实现的接口是否严格匹配契约定义？

人类审查焦点：

自动化验证通过后，人类只需关注自动化无法覆盖的维度：架构一致性（代码是否遵循 ADR？模块是否越界？）、AI 常见问题（是否过度工程化？是否添加了规约外的功能？）、安全性审查。

明确不需要人类关注的：代码格式（交给 Lint）、类型安全（交给 TypeScript）、功能正确性（交给测试）。

五阶段工作流

```
Phase 0          Phase 1          Phase 2          Phase 3       Phase 4
需求理解    →    契约设计    →    规约编写    →    AI 实现    →    验证
                                                                      │
"做什么？       ADR              模块 Spec          组装 prompt      自动化测试
 为什么？"      接口类型         测试规约           AI 生成代码      人类 Review
                 Schema           Prompt 模板        不满意就重来      ──┤
                 边界图                                                 │
                                                                        ▼
                                                                    Phase 5
                                                                    反馈循环
                                                                        │
                                                          AI 反复犯错 → 更新规约
                                                          接口不够 → 更新契约
                                                          Prompt 差 → 迭代模板
                                                          需求变化 → 新 ADR
```

时间分配

这是 CDRE 最反直觉但最重要的特征：

Phase	耗时占比	执行者
Phase 1: 契约设计	40%	人类
Phase 2: 规约编写	30%	人类
Phase 3: AI 实现	10%	AI
Phase 4: 验证	20%	自动化 + 人类

人类 70% 的时间花在"写文档"上，AI 写代码只占 10%。 这颠覆了大多数人对"AI 编程"的想象，但恰恰是 CDRE 的核心主张——前期的契约和规约投入，会在每一次 AI 交互中获得复利回报。

Phase 5：反馈循环的触发条件

反馈循环是 CDRE 持续演进的引擎。以下是常见的触发条件和对应的行动：

观察到的现象	行动
AI 反复生成规约之外的功能	在模块 Spec 的"显式排除"中增加约束
AI 对某个接口的理解总是有偏差	在接口契约中增加注释或收紧类型定义

观察到的现象	行动
测试覆盖了但仍有 bug 漏出	补充边界情况的测试用例
需求发生变化	评估是否需要新的 ADR，更新受影响的契约和规约
Prompt 效果不稳定	迭代 Prompt 模板，增加更明确的指令或示例
新增模块	编写新的模块 Spec、接口契约、测试规约

关键实践详解

实践一：Architecture Decision Records（ADR）

ADR 是 CDRE 契约层的基石。一个好的 ADR 包含：

- **标题**：简明的决策描述
- **Context**：驱动这个决策的背景和约束（业务需求、技术限制、团队状况）
- **Decision**：具体做了什么选择
- **Consequences**：选择带来的正面和负面后果（用 ✅ 和 ⚠️ 标记）

ADR 在 CDRE 中的独特价值在于：当 AI 在某个模块工作时，将相关的 ADR 注入上下文，AI 就能在正确的约束下做出实现决策，而不是按自己的"偏好"来。

经验法则：一个中型项目通常有 3-8 个 ADR。如果你的 ADR 超过 15 个，可能是粒度太细了。

实践二：接口优先（Contract-First）设计

在写任何实现之前，先定义清楚模块之间的接口。推荐的形式化方式：

- REST API → TypeScript 接口定义 或 OpenAPI Spec
- RPC → Protobuf Schema 或 gRPC 定义
- 事件驱动 → 事件类型定义（TypeScript）
- 数据库 → SQL Schema + 约束注释

接口优先的双重好处：AI 可以独立地在每个模块内工作而不会破坏整体一致性；接口定义本身就是最精确的 prompt——它用类型系统表达了"正确的通信方式是什么"。

实践三：测试即规约（Tests as Specifications）

在 CDRE 中，测试的写作顺序是：

1. **E2E 测试**（人类写）——定义业务流程的正确性
2. **集成测试**（人类写）——定义模块边界的正确性
3. **单元测试**（AI 生成）——定义函数级行为的正确性

前两层由人类编写，因为它们编码的是业务知识和架构决策。第三层交给 AI，因为有了契约和规约的约束，AI 可以自动推导出函数级的测试。

关键思维转换：测试不是"写完代码后的质量检查"，而是"写代码前的工作指令"。当你把测试和规约一起注入 prompt，AI 的任务就变成了"让这些测试通过"——这是一个高度明确的、AI 擅长优化的目标。

实践四：版本化的 Prompt 工程

Prompt 模板应当和代码享受同等的工程待遇：

- 存放在版本控制中（如 `prompts/` 目录）
- 有明确的命名约定（如 `implement-api-endpoint.md`、`add-database-migration.md`）
- 记录变更历史和变更原因
- 定期审查和优化

一个好的 Prompt 模板通常包含四个区块：项目上下文（精简版的契约层信息）、模块规约（本次任务相关的 Spec）、测试用例（AI 必须通过的验收标准）、输出要求（格式、风格、约束等）。

实践五：显式排除（Explicit Exclusions）

AI 有天然的"过度工程化"倾向——如果你不告诉它"不要做什么"，它倾向于添加各种"改进"。在每个模块规约中明确列出"不做什么"，是 CDRE 中最实用的技巧之一。

典型的显式排除包括：当前阶段不需要的功能（如 MVP 阶段不做认证）、超出模块职责的功能、优化类需求（如"不做缓存"）。

与传统方法论的关系

CDRE 不是凭空发明的，它继承了经典软件工程中经过时间检验的思想，并在 AI 时代的新约束下重新组合了它们。

V-Model 的结构：每一层设计都有对应的一层验证——系统设计对应 E2E 测试，模块设计对应集成测试，详细设计对应单元测试。CDRE 的四层架构本质上复用了这一映射关系。

螺旋模型的迭代节奏：CDRE 不是一次性写完所有文档再开始开发，而是每轮迭代中逐步细化契约、扩展实现、收集反馈。

Design by Contract 的哲学：Bertrand Meyer 提出的"契约式设计"——模块间通过前置条件、后置条件和不变量来定义契约——正是 CDRE 接口优先策略的思想基础。

与传统方法论的根本区别在于成本结构的颠覆：传统方法论优化的是"减少编码返工的成本"，CDRE 优化的是"提高 AI 产出质量的精度"。编码成本趋近于零后，文档从"沟通工具"升级为"编程接口"——其精度直接决定了系统的质量上限。

方法论对比

维度	传统敏捷 (Scrum)	CDRE
核心循环	2 周 Sprint	10 分钟 生成→验证
文档定位	沟通工具，"够用就好"	编程接口，是核心资产
代码定位	核心资产，精心维护	一次性产物，随时可重新生成
测试定位	事后验证	事前规约
人类角色	编码者	架构师 + 规约设计者 + 验证者
AI 角色	辅助工具	核心实现者
沟通成本	人际对齐（$O(n^2)$）	文档精度（线性可控）
质量保证	Code Review 为主	自动化验证为主 + 架构 Review
迭代方式	修改现有代码	重新生成代码
技术债务	逐渐积累	理论上可通过重新生成消除

适用场景与局限性

适合使用 CDRE 的场景

- 中大型项目（多模块、持续迭代数月以上）
- 有明确的模块边界和接口的系统
- AI 能力覆盖的技术栈（主流 Web、API、数据库等）
- 需要长期维护和演进的产品

不适合的场景

- 一次性脚本或小工具（直接 vibe coding 更快）
- 高度探索性的原型（需求本身还不清晰，无法写契约）
- AI 能力尚不覆盖的领域（如特定嵌入式系统、冷门语言）

已知局限

- 契约层的质量完全依赖人类的架构能力——如果架构设计有误，AI 会高效地实现一个错误的系统
- 当需求频繁剧烈变化时，契约层的维护成本可能抵消 AI 实现的效率收益
- 目前缺乏成熟的工具链来自动化"契约→规约→prompt"的组装过程

快速上手

如果你想在自己的项目中尝试 CDRE：

第一步（30 分钟）：为你的项目写 1-2 个 ADR，回答"为什么用这个技术栈"和"最关键的设计决策是什么"。

第二步（1 小时）：用 TypeScript 或其他形式化方式定义模块间的接口。不写实现，只写类型定义。

第三步（30 分钟）：为最核心的模块写一份不超过一页的 Spec，包含：职责、依赖、行为规则、显式排除。

第四步（30 分钟）：为这个模块写 3-5 个测试用例，覆盖正常路径和主要错误路径。

第五步（10 分钟）：组装 prompt（契约 + 规约 + 任务 + 测试），让 AI 生成实现，观察产出质量。

第六步（5 分钟）：运行测试。通过则完成一个循环；未通过则调整规约或 prompt，重新生成。

总耗时约 3 小时，完成第一个 CDRE 循环。

常见问题

Q: 花这么多时间写文档，不是比直接让 AI 写代码更慢吗?

对于一次性的小脚本确实如此。CDRE 的投资回报率在规模化时显现——当你的项目有 10 个模块、需要持续迭代数月时，前期的契约投入会在每一次 AI 交互中获得复利回报。就像类型系统对小项目是负担、对大项目是救命稻草一样。

Q: 契约需要多详细?

遵循"AI 犯错就加约束"的渐进原则。一开始写得粗一些，发现 AI 反复在某个点出错，就把那个点的约束写进规约。这是一个逐步收敛的过程，不需要一次性做到完美。

Q: 如果 AI 能力继续提升，CDRE 还有意义吗?

AI 的上下文窗口变大、推理能力变强，会改变模块分解的粒度（模块可以更大），但不会消除对契约的需求。只要系统复杂度超过 AI 的单次理解能力（无论那个能力有多强），就需要人类来定义边界。这就像操作系统不会因为 CPU 变快就不需要进程隔离一样——隔离解决的不是性能问题，而是复杂性管理问题。

Q: 多人协作时 CDRE 怎么运作?

契约层和规约层由团队共同维护（类似于团队共同维护数据库 Schema）。不同成员可以同时用 AI 开发不同模块，因为接口契约保证了模块间的兼容性。协作的沟通开销从"对齐彼此的代码实现"降低为"对齐契约定义"——后者的变化频率远低于前者。

Q: CDRE 和 TDD 的关系是什么?

CDRE 包含了 TDD 的"测试先行"思想，但范围更大。TDD 关注的是函数级的"红→绿→重构"循环，CDRE 将这一思想提升到系统级——契约先行、规约先行、测试先行，然后 AI 来"让所有测试变绿"。

CDRE v0.1 — 这份文档本身也欢迎迭代。
It's contracts all the way down.